Ensemble Learning on the Adult Income Census Dataset

Random Forest, AdaBoost, and XGBoost for Income Classification

Dataset: Adult Income Census Dataset (Kaggle: https://www.kaggle.com/datasets/uciml/adult-census-income)

1 Problem Statement

The goal of this project is to predict whether an individual's annual income exceeds $50,000 based on demographic and employment-related attributes such as age, education, occupation, marital status, and hours worked per week. This is a binary classification problem (*income* $\leq$ \$50K vs. $>$ \$50K) built on the UCI Adult Income Census dataset. The project compares three ensemble learning methods – one bagging-based (Random Forest) and two boosting-based (AdaBoost, XGBoost) – to identify which approach best balances predictive accuracy, robustness, and interpretability for this task, and to understand which socioeconomic factors are most predictive of higher income.

2 Dataset Overview

The dataset contains **32,561 records** and **15 attributes**, combining numerical and categorical features that describe an individual's demographic and employment profile.

- **Numerical features:** `age`, `fnlwgt`, `education.num`, `capital.gain`, `capital.loss`, `hours.per.week`
- **Categorical features:** `workclass`, `education`, `marital.status`, `occupation`, `relationship`, `race`, `sex`, `native.country`
- **Target variable:** `income` – binary (`<=50K` / `>50K`)

Target distribution. The dataset is moderately imbalanced:

Class	Count	Percentage
$\leq$ \$50K	24,698	75.91%
$>$ \$50K	7,839	24.09%

Table 1: Distribution of the target variable

Data quality. No columns contained standard `NaN` values initially, but missing entries were encoded with the placeholder string `"?"`. After converting these to true missing values, three columns were affected: `workclass` (1,836 missing), `occupation` (1,843 missing), and `native.country` (583 missing). The dataset also contained **24 exact duplicate rows**, which were removed, leaving **32,537 records**.

3 Data Preprocessing

The following preprocessing steps were applied before model training:

1. **Missing value handling:** Placeholder `"?"` values were converted to `NaN`, and missing entries in `workclass`, `occupation`, and `native.country` were imputed using the mode (most frequent category) of each column.
2. **Duplicate removal:** 24 duplicate rows were dropped.
3. **Feature dropping:** `fnlwgt` (a census sampling weight with no individual predictive meaning) and `education` (redundant with the numeric `education.num`) were removed.
4. **Feature engineering:** A new feature, `capital.net` (`capital.gain` − `capital.loss`), was created to capture net investment income in a single variable.
5. **Target encoding:** `income` was mapped to binary labels (`0` = $\leq$\$50K, `1` = $>$\$50K).
6. **Categorical encoding:** One-Hot Encoding (`drop="first"`, with `handle_unknown="ignore"`) was applied to all categorical features.

7. **Feature scaling:** Numerical features were standardized using `StandardScaler`.
8. **Class imbalance:** Rather than oversampling, the imbalance was handled by (a) stratified train/test splitting to preserve class proportions, and (b) evaluating models primarily on F1-score and ROC-AUC rather than raw accuracy.
9. **Train/test split:** An 80:20 stratified split produced **26,029** training records and **6,508** test records.

All transformations were wrapped in a single `scikit-learn Pipeline` (`ColumnTransformer` + classifier) to prevent data leakage between the train and test sets.

4 EDA Findings

Exploratory analysis of the numerical and categorical distributions, correlations, and target relationships revealed the following patterns:

- The target classes are imbalanced (roughly 3:1 in favor of $\leq$\$50K), which motivated the use of F1-score and ROC-AUC as primary evaluation metrics.
- Numerical features such as `capital.gain` and `capital.loss` are heavily right-skewed, with most individuals reporting zero.
- `age` and `hours.per.week` show a visible positive association with higher income: individuals earning >\$50K skew older and tend to work more hours per week.
- Certain categories dominate the categorical features (e.g., `workclass = Private`, `native.country = United-States`), which informed the choice of one-hot encoding with a "drop first" strategy to avoid the dummy-variable trap.

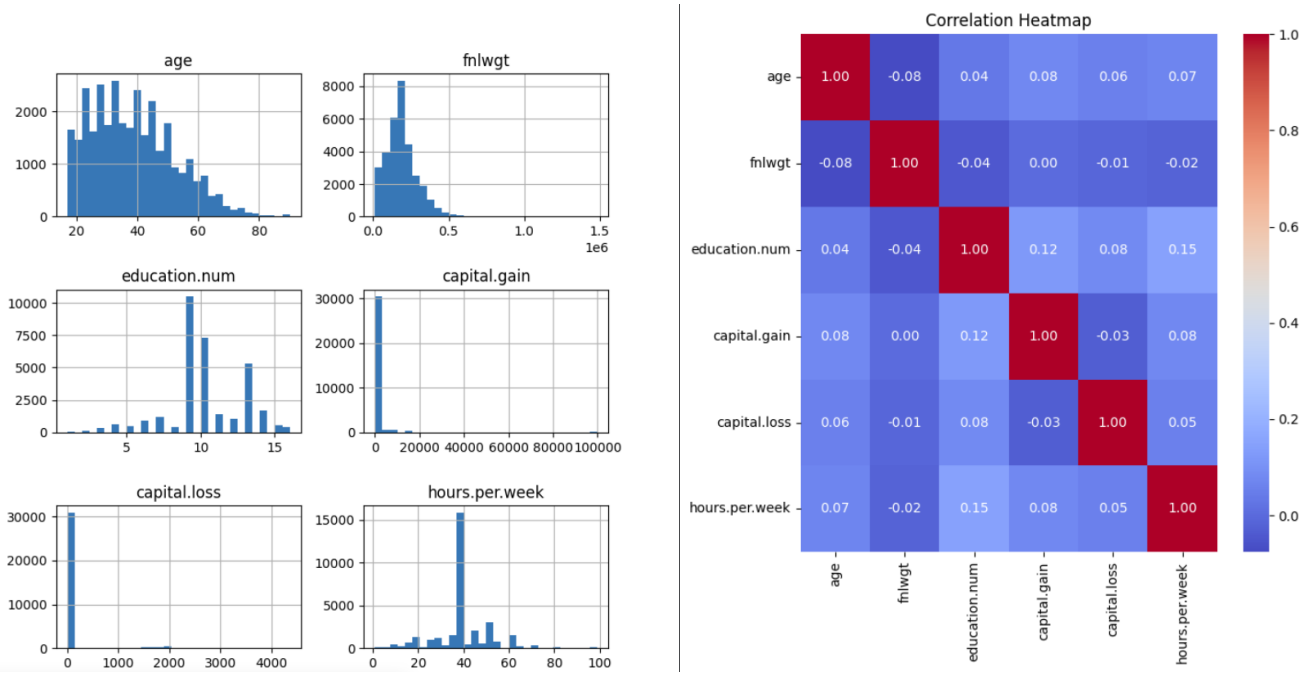

Figure 1: Distribution of numerical features (left) and their correlation heatmap (right).

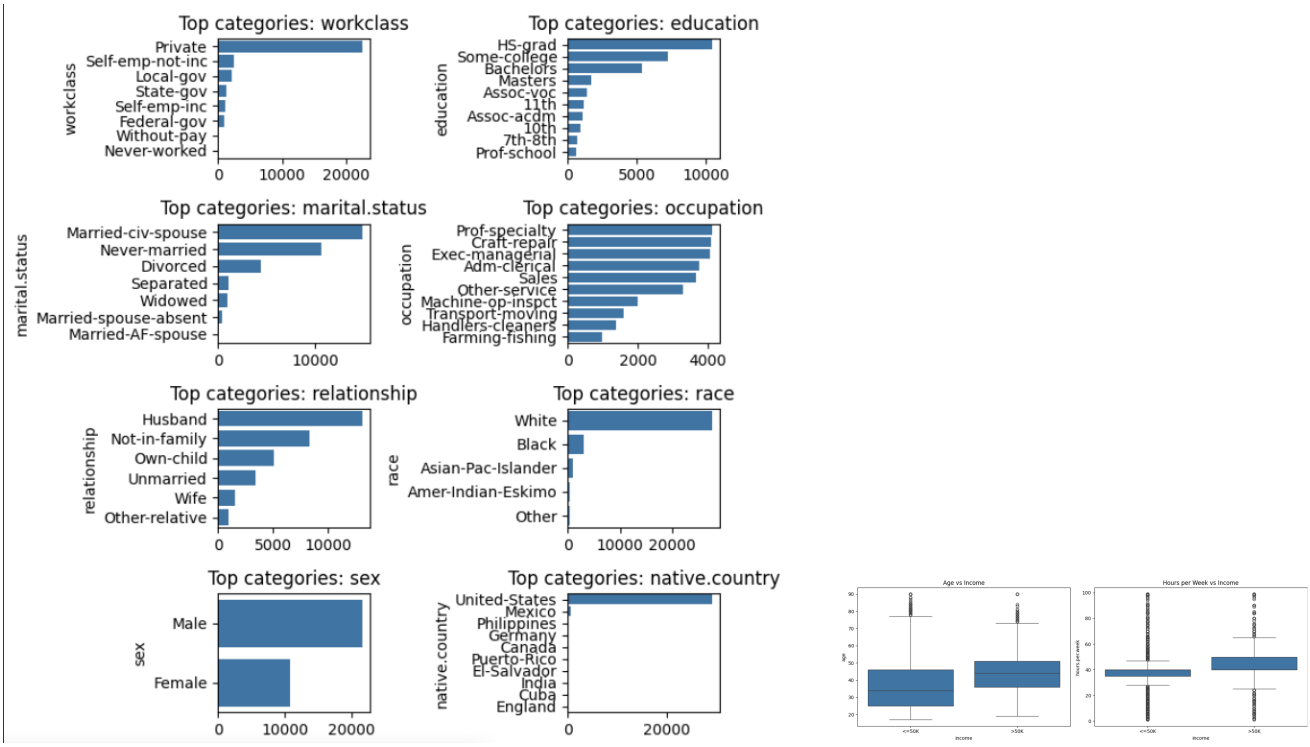

Figure 2: Top categories within key categorical features (left) and age/hours-per-week distributions split by income class (right).

5 Model Building

Three ensemble classifiers were implemented, each wrapped in a preprocessing + classifier `Pipeline`:

- **Random Forest Classifier** – a bagging ensemble of decision trees trained on bootstrapped samples with random feature subsets at each split.
- **AdaBoost Classifier** – a boosting ensemble that sequentially trains weak learners (decision stumps by default), reweighting misclassified samples at each iteration.
- **XGBoost Classifier** – a regularized gradient boosting framework that builds trees sequentially to minimize a second-order approximation of the loss function.

Each model was trained inside a `GridSearchCV` (3-fold cross-validation, optimizing for F1-score) to jointly select hyperparameters and fit the final model on the training set, then evaluated on the held-out 20% test set.

6 Evaluation Results

Table 2 summarizes test-set performance for all three tuned models.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
XGBoost	0.8677	0.7734	0.6378	0.6991	0.9221
Random Forest	0.8594	0.7793	0.5810	0.6657	0.9131
AdaBoost	0.8523	0.7871	0.5306	0.6339	0.9026

Table 2: Test-set evaluation metrics (sorted by F1-score).

XGBoost achieved the best overall performance, leading on accuracy (86.77%), F1-score (0.699), and ROC-AUC (0.922). Random Forest was a close second, while AdaBoost had the highest precision of the three but the lowest recall, indicating it was more conservative in predicting the >$50K class.

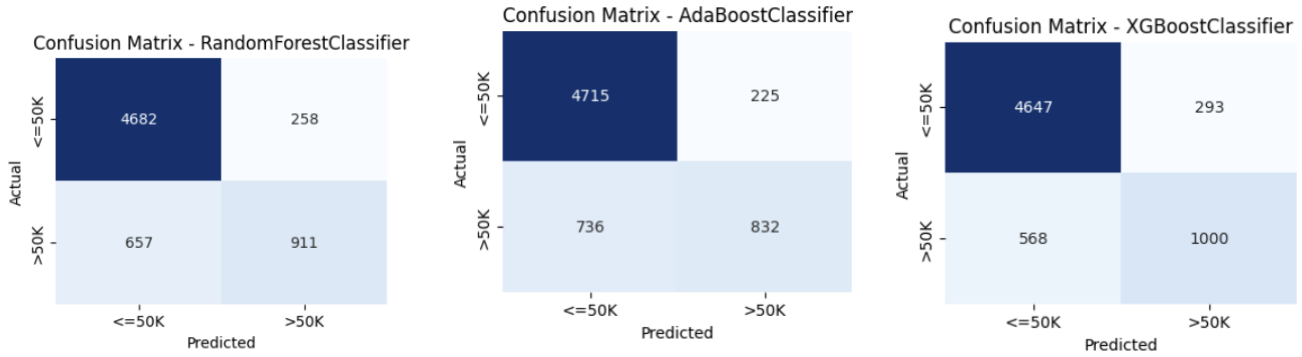

Figure 3: Confusion matrices for Random Forest, AdaBoost, and XGBoost (left to right).

Across all three models, the confusion matrices show the same pattern typical of imbalanced classification: the majority class ($\leq$\$50K) is predicted with high accuracy, while a larger share of the minority class (>\$50K) is missed (false negatives), which is reflected in the comparatively lower recall scores.

7 Hyperparameter Tuning Results

Hyperparameter tuning was performed with `GridSearchCV` (3-fold CV, F1-score as the scoring metric) over the grids below.

Model	Grid Searched	Best Parameters
Random Forest	n_estimators: [100, 200] max_depth: [10, 20] min_samples_leaf: [1, 2]	n_estimators = 100 max_depth = 20 min_samples_leaf = 1
AdaBoost	n_estimators: [50, 100, 200] learning_rate: [0.01, 0.1, 0.5]	n_estimators = 200 learning_rate = 0.5
XGBoost	n_estimators: [100, 200] max_depth: [3, 5] learning_rate: [0.01, 0.1]	n_estimators = 200 max_depth = 5 learning_rate = 0.1

Table 3: Hyperparameter grids and best configurations found via GridSearchCV.

Did tuning help? All three models used their best cross-validated configuration directly, so the results in Table 2 already reflect tuned performance. The key observations from the search itself:

- **Random Forest** preferred a shallower estimator count (100 trees) but a deeper max depth (20), and benefited from allowing leaves as small as 1 sample, suggesting the trees needed enough depth to capture interactions without requiring an unusually large forest.
- **AdaBoost** performed best with the highest learning rate tested (0.5) and the most estimators (200), indicating that more aggressive, longer boosting helped it fit the data better than the conservative default settings – though it still lagged behind the other two models on recall and F1.
- **XGBoost** selected a moderate depth (5) and learning rate (0.1) with 200 estimators, consistent with gradient boosting's typical preference for many shallow trees with regularized learning rather than a few complex ones – this configuration produced the best overall result.

In general, boosting models (AdaBoost, XGBoost) were more sensitive to their hyperparameter choices than Random Forest, whose bagging structure makes it inherently more robust to sub-optimal settings.

8 Feature Importance Analysis

Feature importances were extracted from the tuned Random Forest and XGBoost models (AdaBoost's default weak learners make its per-feature importances less stable and are omitted here).

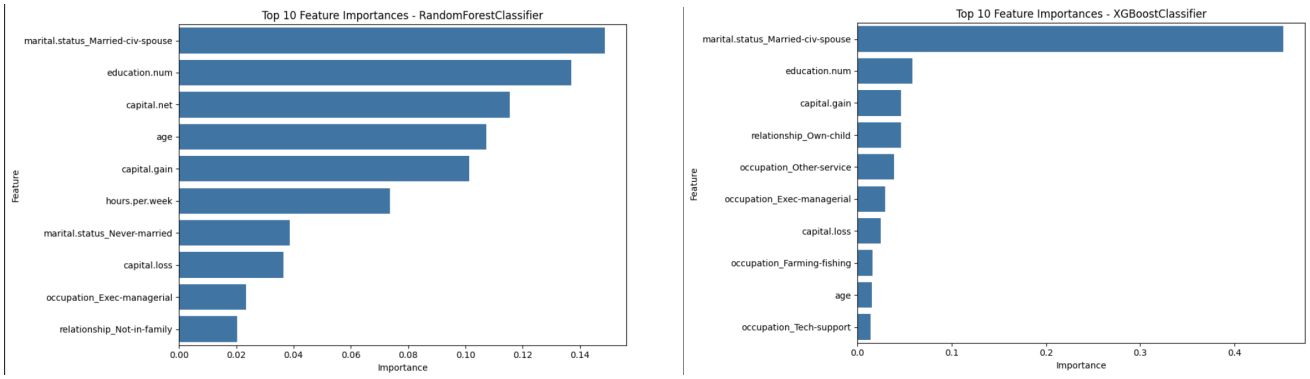

Figure 4: Top 10 feature importances – Random Forest (left) and XGBoost (right).

Why these features matter:

- **marital.status_Married-civ-spouse** – Being married to a civilian spouse often correlates with higher household income, potentially due to combined financial planning and dual-income households.
- **education.num** – Represents years of education; higher education levels typically unlock better-paying job opportunities.
- **capital.gain / capital.loss** – Higher capital gains (or lower capital losses) often correlate with higher income, reflecting investment activity that is more common among higher earners.
- **relationship (various categories)** – Similar to marital status, these categories are highly indicative, as certain roles tend to come with established careers and higher earning potential within a household.
- **occupation (various categories)** – Certain occupations inherently pay more than others, so dummy features representing these high-paying professions carry high importance.
- **hours.per.week** – Hours worked directly influences income, especially for hourly wage earners or those receiving overtime pay.
- **age** – Income tends to rise with age as individuals gain experience and advance in their careers, typically peaking before retirement.

9 Final Comparison

9.1 Which model performed best, and why?

XGBoost was the best-performing model on every headline metric (accuracy, F1-score, and ROC-AUC). This is consistent with general expectations for gradient boosting on structured/tabular data: XGBoost uses second-order gradient information and built-in L1/L2 regularization to fit residual errors more precisely than AdaBoost's simpler reweighting scheme, while also correcting bias more effectively than Random Forest's variance-reduction-only bagging strategy.

9.2 Bagging vs. Boosting

Aspect	Random Forest (Bagging)	AdaBoost (Boosting)	XGBoost (Boosting)
Accuracy	85.94% – strong, stable baseline	85.23% – lowest of the three	86.77% – best overall
Training time	Fast; trees built independently/in parallel	Fast per estimator, but sequential (no parallelism across estimators)	Slower per tree, but highly optimized and scales well
Interpretability	Moderate (feature importances over many trees)	Higher (few, simple stumps)	Moderate (feature importances, compatible with SHAP)
Robustness	High – averaging over trees reduces variance and limits overfitting	Lower – sensitive to noisy/mislabeled data since it upweights misclassified points	High – regularization, shrinkage, and subsampling control overfitting

Table 4: Bagging vs. Boosting comparison based on experimental results.

9.3 Strengths and Limitations

- **Random Forest** – *Strengths:* robust, low variance, minimal tuning required, trains in parallel. *Limitations:* can be memory-heavy with many deep trees; importance scores can be biased toward high-cardinality categorical features.
- **AdaBoost** – *Strengths:* simple, interpretable, fast to train. *Limitations:* sensitive to noisy data and outliers; in this experiment it had the lowest recall and F1-score, missing more of the >\$50K class than the other models.
- **XGBoost** – *Strengths:* regularization (L1/L2), handles missing values internally, best accuracy/F1/ROC-AUC in this study. *Limitations:* more hyperparameters to tune, longer training time per model, and lower interpretability than a single decision tree.

10 Key Learnings and Conclusion

This project implemented and compared three ensemble learning algorithms – Random Forest, AdaBoost, and XGBoost – on the Adult Income Census dataset to predict whether an individual earns more than \$50K per year.

Key learnings:

- Careful preprocessing (missing-value imputation, redundant-feature removal, encoding, and feature engineering such as `capital.net`) has a direct impact on downstream model quality, and wrapping these steps in a `Pipeline` prevents data leakage between train and test sets.
- On a moderately imbalanced target, accuracy alone is not a reliable metric; F1-score and ROC-AUC give a more complete picture, and both consistently favored XGBoost in this study.
- Boosting methods are more sensitive to hyperparameter choices than bagging: AdaBoost's performance depended heavily on learning rate and estimator count, while Random Forest performed well across a wider range of settings.
- Feature importance analysis confirmed that marital status, education level, capital gains/losses, occupation, hours worked, and age are the strongest drivers of income in this dataset – findings that align well with real-world socioeconomic intuition.

Conclusion: Among the three ensemble methods evaluated, **XGBoost** is the recommended model for this task, offering the best balance of accuracy (86.77%), F1-score (0.699), and ROC-AUC (0.922). Random Forest remains a strong, low-maintenance alternative when training simplicity and robustness are prioritized over squeezing out the last few points of performance, while AdaBoost – though fast and interpretable – underperformed the other two ensembles on this particular dataset.